CI at Uber in the era of AI

Buildbarn Community Meetup, October 2026

Uber

Hi, I'm Eaugene

Software Engineer, Developer Platform at Uber

I work on CI Infra Team managing remote build execution and caching — the Buildbarn deployment that sits behind Uber's CI.

Before Uber, I was at D. E. Shaw Hedge Fund.

Most of my week comes down to a single question: how do you keep a build fast while the amount of code arriving at it keeps doubling?

Away from the terminal

Long walks

And cooking fusion food - The success rate is improving.

Agenda

01 The end of easy compute

02 Where Buildbarn fits in Uber

03 The efficiency we actually got

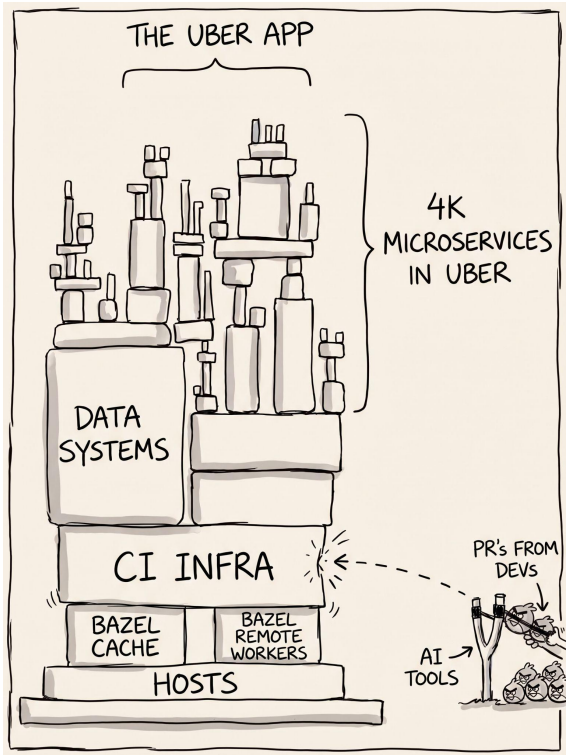
04 Software Factory & Buildbarn

05 Two experiments

01

The end of easy compute

What all of it rests on



With apologies to xkcd #2347

The block at the bottom

The developers arrive a great deal faster than they used to.

Uber CI

One monorepo per language

Go, Java, Python, Android, iOS and Web — thousands of services, with a single build graph inside each repository. Bazel drives them.

A CI orchestration layer

Every push, pull request and merge-queue attempt becomes a CI job on an agent.

Bazel Cache

Hosted with Buildbarn

What changed: code started arriving faster than people type

Agents open pull requests

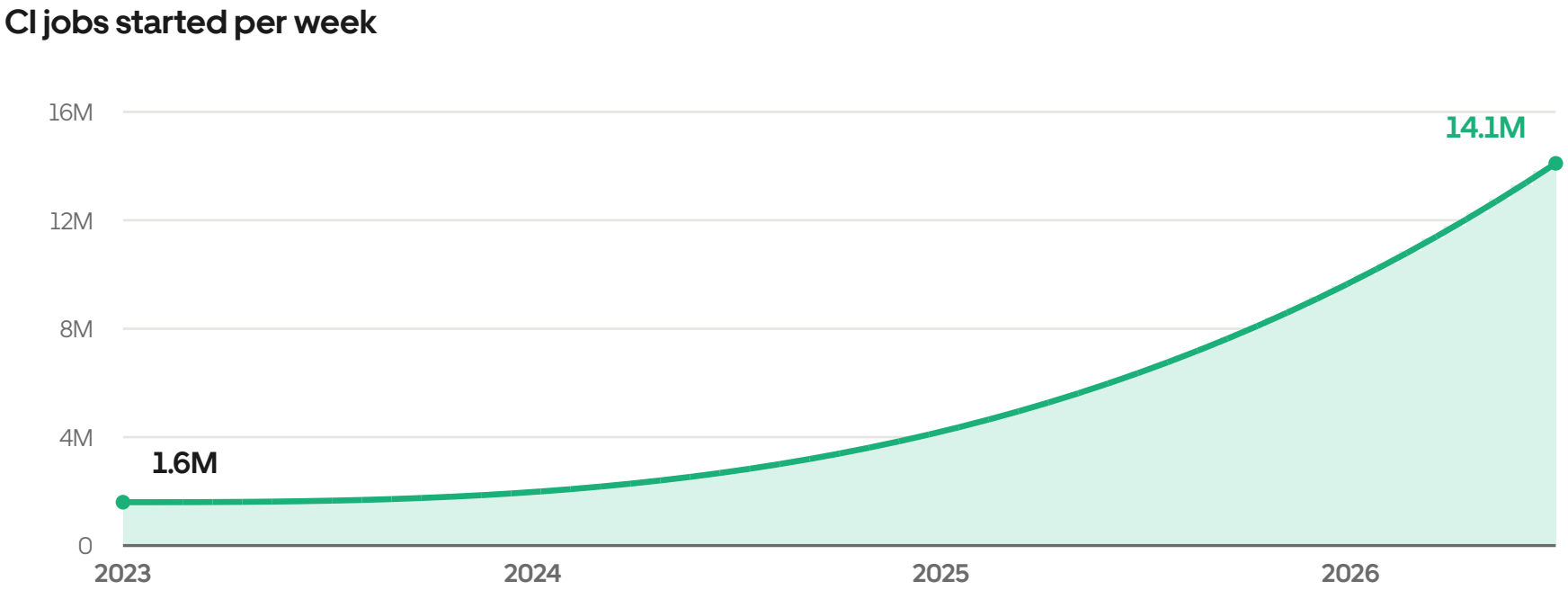
Iteration got cheap for humans, not for CI

The load stopped following office hours

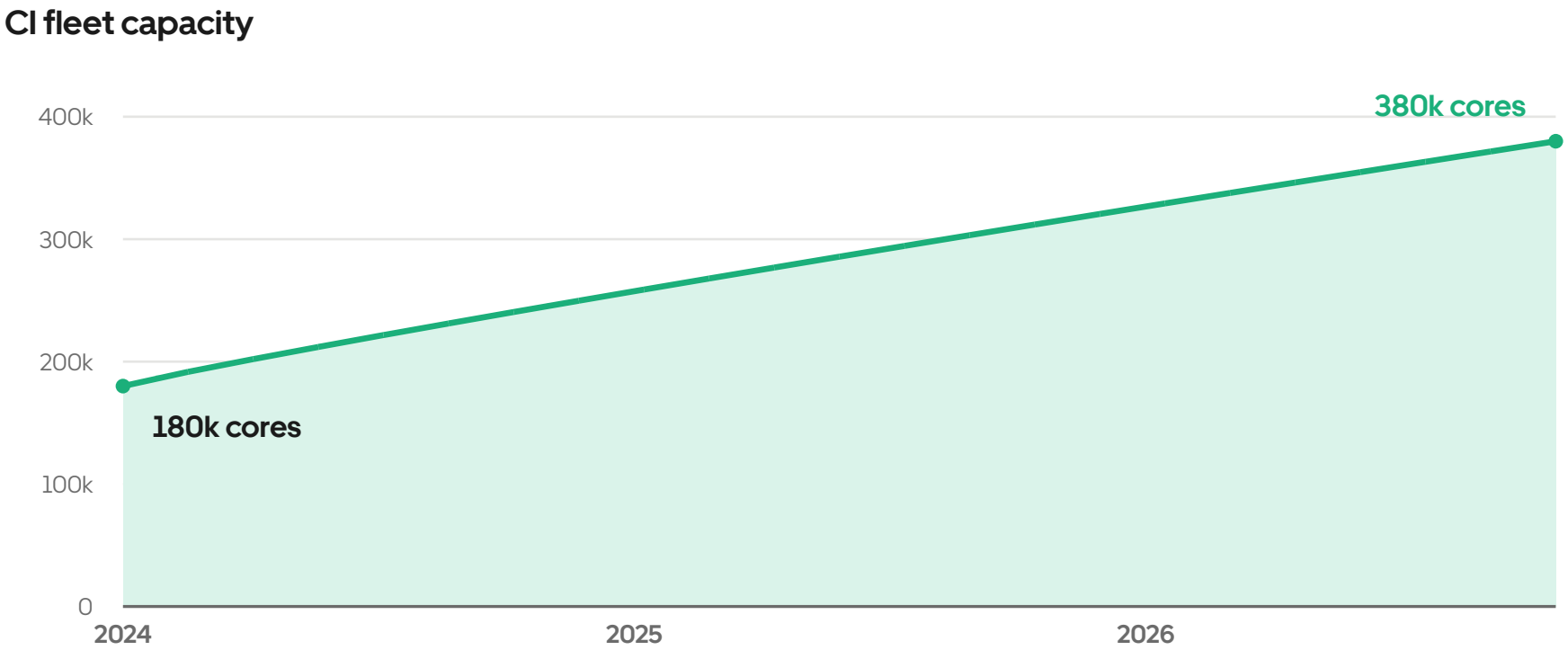
Demand: 1.6M → 14.1M CI jobs a week



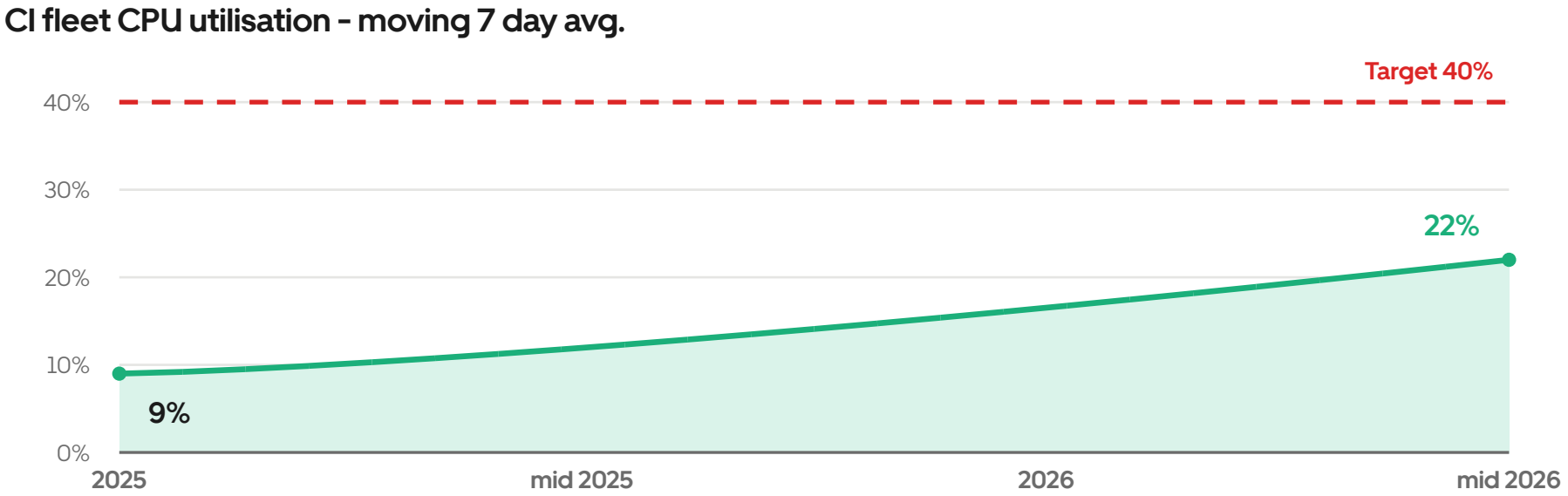
Demand: 1.6M → 14.1M CI jobs a week



The old answer: buy more cores

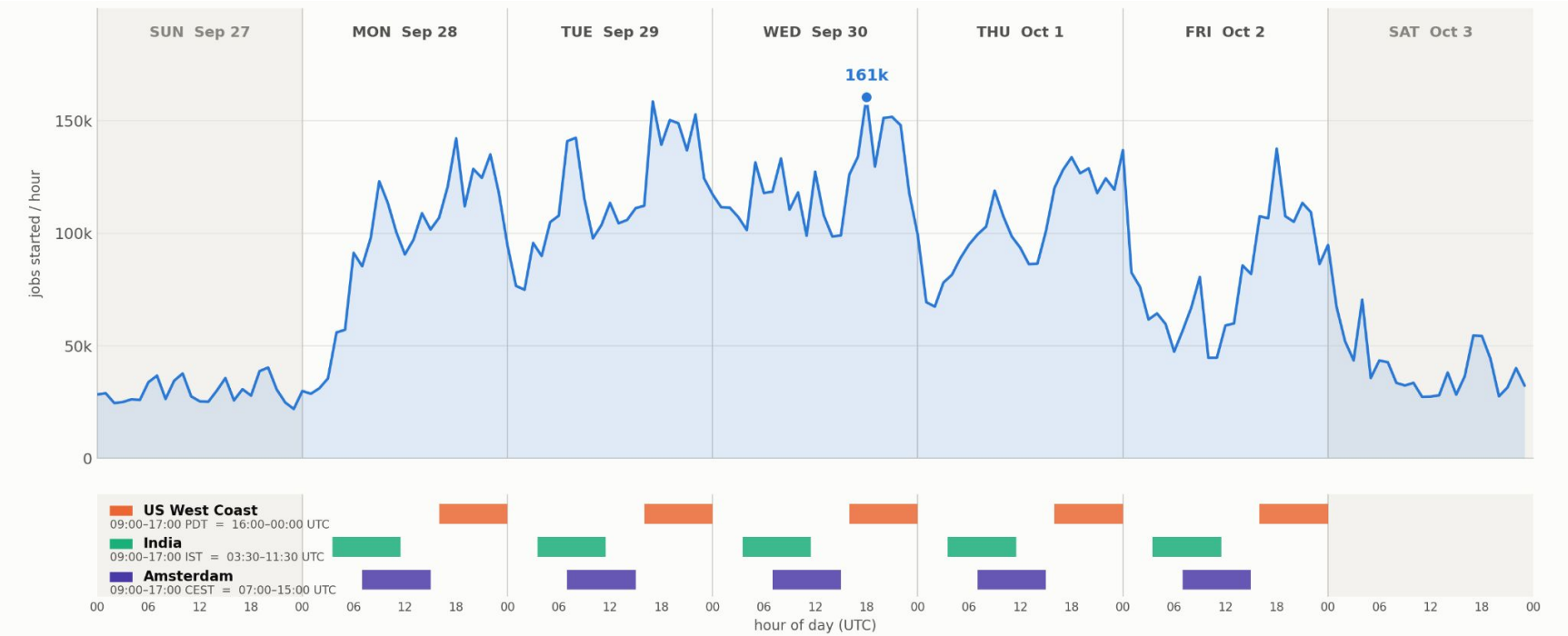


Improving Utilisation: 9% → 22%.

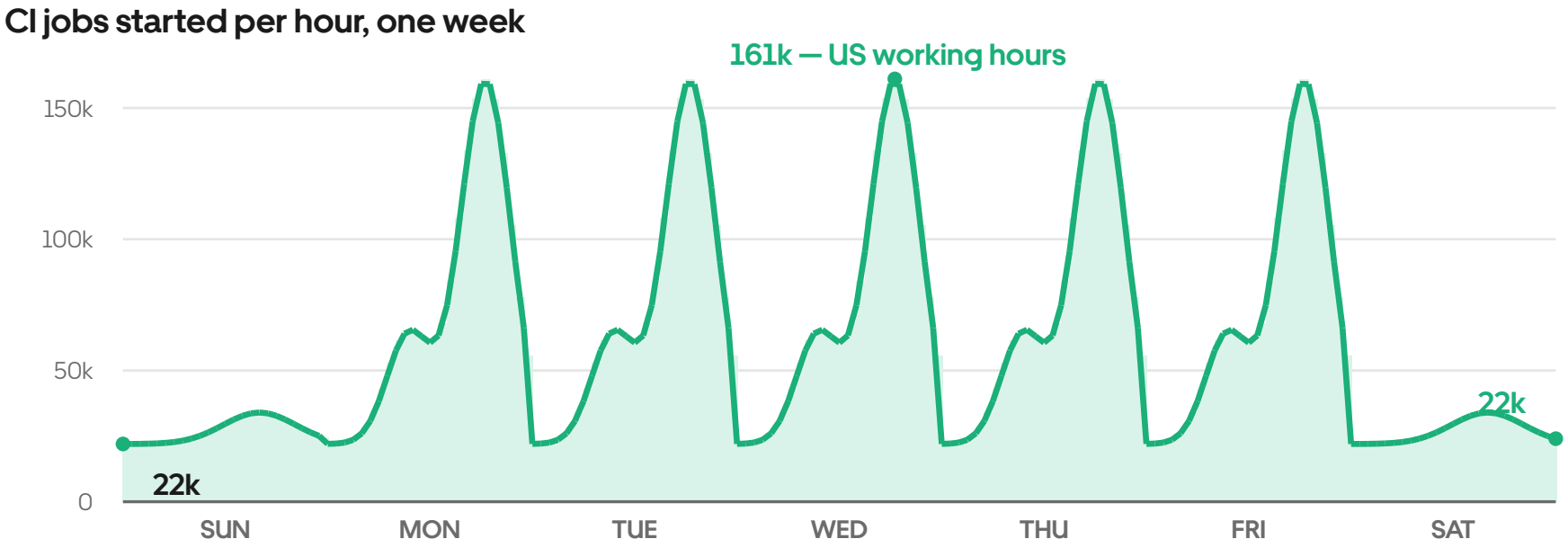


Through better bin-packing and job shaping - brought the utilisation from 9% (in start of 2025) to 22% as of mid 2026. More work is in flight on resource-aware scheduling of CI jobs.

Demand swings 7× in a single day



Demand swings 7× in a single day



Every weekday peak sits inside US working hours, 16:00–00:00 UTC. The troughs are nights and weekends — the same fleet, idle.

02

Where Buildbarn fits in Uber

CI - 2 regions & Developers - 5 regions



CI and developers share one warm cache

In the US

CI and developers hit the same cache. Every pull-request run writes into it, so the cache a developer reads has already been warmed by CI — nobody pays to warm it separately.

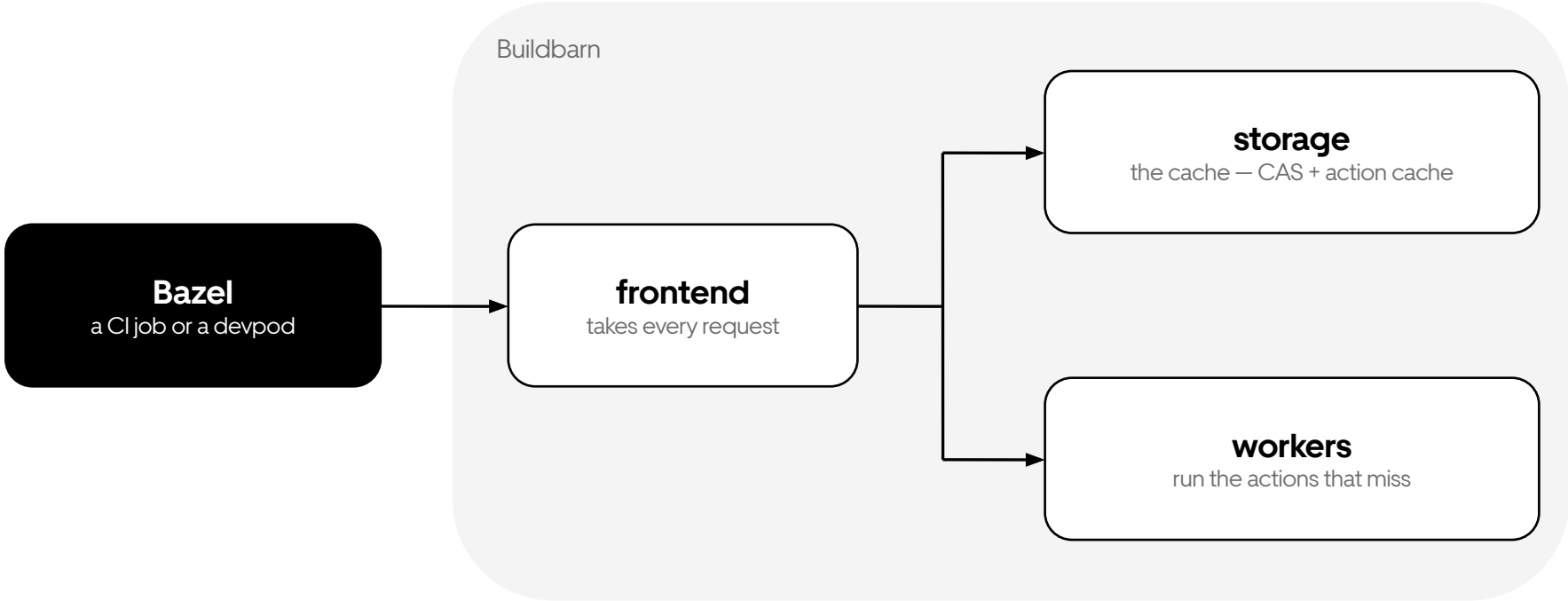
In every other region

Developers get a cache hosted in their own region, so a read stays local instead of crossing an ocean and coming back for every miss.

Keeping those warm

There is no CI traffic out there to do the warming, so a periodic Bazel build runs against each regional cache and populates it ahead of the developers.

Buildbarn, in three parts



03

The efficiency we actually got

What Buildbarn bought us

20%

more compute-efficient CI overall

40%

more elastic RBE capacity — workers autoscale against queue demand instead of standing at a peak-sized count

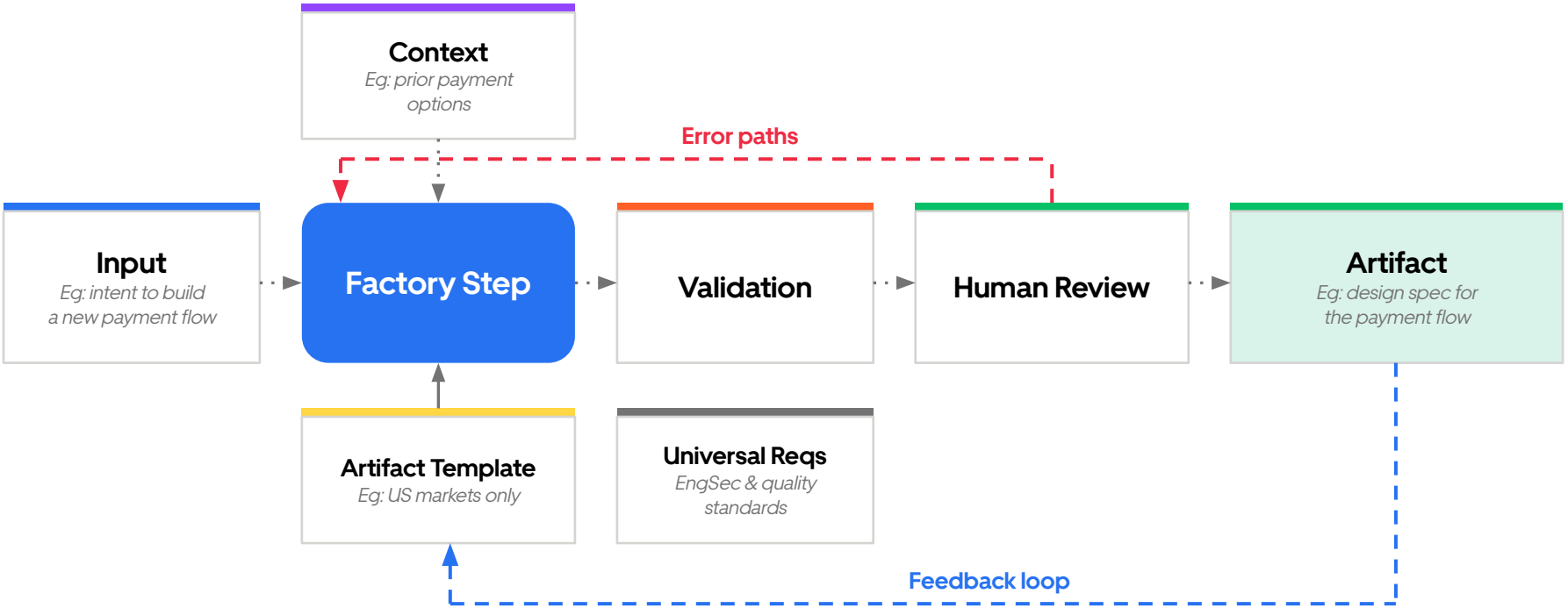
20 min

land time in the monorepo, still inside its SLA with Buildbarn carrying the work — even as AI load kept climbing

04

Software Factory & Buildbarn

The Software Factory, one step expanded



Brains and hands

Traditional — one high-resource machine per developer



The machine sits idle most of the time, because most of the time the developer is typing.

With Buildbarn — brains stay local, hands are shared



One pool of high-resource workers absorbs every developer's builds, so many build machines can be spun up in software factory .

05

Two experiments

What we want to try next

1 · Frontend + storage per node

Pin one frontend and one storage shard per node . both carving logical volumes out of that node's NVMe. The network hop on a local miss disappears.

(Thanks Ed for suggesting this in office hours)

2 · One cluster for CI + Buildbarn

CI demand swings 7× a day. Buildbarn's swings on a different curve. Put both on the same nodes and each borrows the other's trough

Thank you

Questions

Uber